

```

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/kdev_t.h>

static int major;
static char *name = "null_driver";

static int null_open(struct inode *i, struct file *f)
{
    printk(KERN_INFO "Calling: %s\n", __func__);
    return 0;
}

static int null_release(struct inode *i, struct file *f)
{
    printk(KERN_INFO "Calling: %s\n", __func__);
    return 0;
}

static ssize_t null_read(struct file *f, char __user *buf,
size_t len, loff_t *off)
{
    printk(KERN_INFO "Calling: %s\n", __func__);
    return 0;
}

static ssize_t null_write(struct file *f, const char __user
*buf, size_t len, loff_t *off)
{
    printk(KERN_INFO "Calling: %s\n", __func__);
    return len;
}

static struct file_operations null_ops =
{
    .owner    = THIS_MODULE,
    .open     = null_open,
    .release  = null_release,
    .read     = null_read,
    .write    = null_write
};

static int __init null_init(void)
{
    major = register_chrdev(0, name, &null_ops);
    if (major < 0) {
        printk(KERN_INFO "Failed to register driver.");
        return -1;
    }

    printk(KERN_INFO "Device registered successfully.\n");
    return 0;
}

```

```

static void __exit null_exit(void)
{
    unregister_chrdev(major, name);
    printk(KERN_INFO "Device unregistered successfully.\n");
}

module_init(null_init);
module_exit(null_exit);

MODULE_AUTHOR("Narendra Kangraikar.");
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Null driver");

```

Our driver code is ready. Let us compile and insert the module. In the article last month, we did learn how to write *Makefile* for kernel modules.

```
[mickey]$ make
```

```
[root]# insmod ./null_driver.ko
```

We are now going to create a device file for our driver. But for this we need a major number, and we know that our driver's `register_chrdev()` function will allocate the major number dynamically. Let us find out this dynamically allocated major number from `/proc/devices`, which shows the currently loaded kernel modules:

```
[root]# grep "null_driver" /proc/devices
248 null_driver
```

From the above output, we are going to use '248' as a major number for our driver. We are only interested in the major number, and the minor number can be anything within a valid range. I'll use '0' as the minor number. To create the character device file, use the `mknod` utility. Please note that to create the device file you must have superuser privileges:

```
[root]# mknod /dev/null_driver c 248 0
```

Now it's time for the action. Let us send some data to the pseudo device using the `echo` command and check the output of the `dmesg` command:

```
[root]# echo "Hello" > /dev/null_driver

[root]# dmesg
Device registered successfully.
Calling: null_open
Calling: null_write
Calling: null_release

```

Yes! We got the expected output. When *open*, *write*, *close*